STRUCTURAL DESIGN PATTERN

Adapter Design Pattern in C#

Complete technical guide: Intent, Real-World Friction, UML Structure, C# Implementations, and Framework Trade-offs.

1. Intent & Core Problem

The **Adapter Pattern** (also called a *Wrapper*) is a structural pattern that allows objects with incompatible interfaces to collaborate. It acts as an intermediary translator that captures calls from a client and converts them into calls that an existing or third-party service can understand.

⚠ The Real-World Problem: Incompatible Third-Party Payment API

An enterprise invoicing application relies on an internal interface `IPaymentProcessor` requiring a standard method: `ProcessPayment(string invoiceId, decimal amount)`.

The business needs to integrate a legacy or external provider (e.g., **LegacyBankSystem**) whose API requires currency conversion, integer cents, and token exchange through a completely different contract: `SendWire(long cents, string destinationAccount, string authToken)`.

Anti-Pattern: Modifying existing client business logic or polluting domain controllers with conditional logic and data conversion branches violates the Open/Closed Principle (OCP) and tightly couples the domain to third-party vendor details.

❌ Before: Leaking Vendor Types into Domain Services

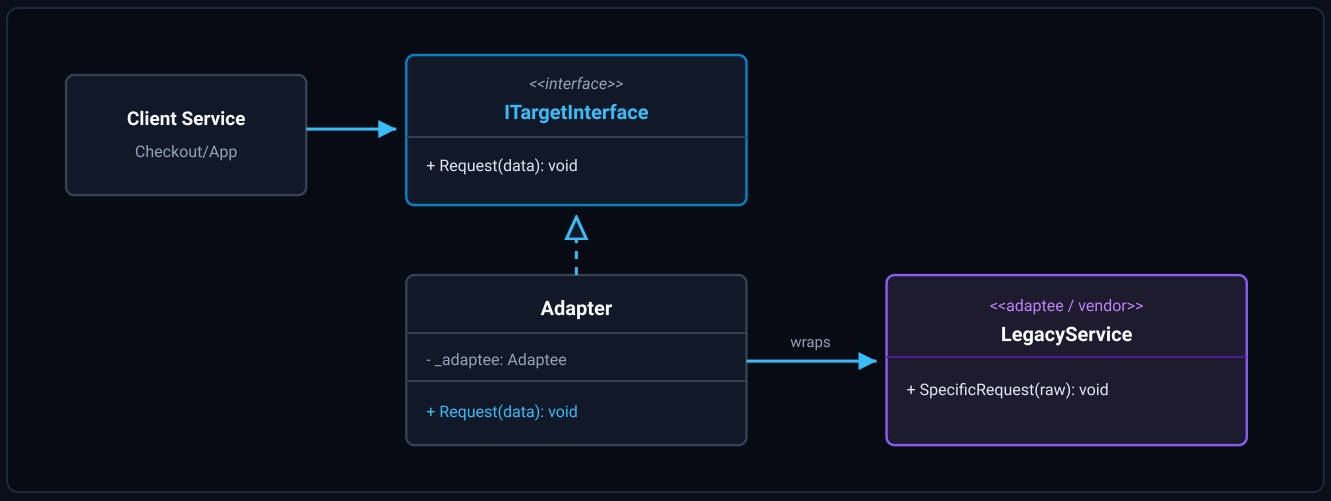
```
// Domain code becomes polluted with custom conditionals, conversions, and vendor SDKs
public class CheckoutService {
    public void CompleteOrder(string vendor, string orderId, decimal amount) {
        if (vendor == "LegacyBank") {
            var legacy = new LegacyBankSystem();
            long cents = (long)(amount * 100);
            legacy.SendWire(cents, "ACC-9901", "TEMP_TOKEN_XYZ");
        } else {
            // standard processing...
        }
    }
}
```

✅ The Solution: Object Adapter Wrapper

Create an **Adapter** that implements your application's domain interface (`IPaymentProcessor`) and wraps an instance of the third-party component (*Adaptee*). The client talks exclusively to the domain interface, unaware of the underlying vendor details.

2. UML Architecture & Structure

In modern C#/.NET, the **Object Adapter** (composition) is favored over the Class Adapter (multiple inheritance) because C# does not support multi-class inheritance and favors composition over inheritance.



3. Structural Component Responsibilities

Role	Class / Interface	Responsibility
Target	`IPaymentProcessor`	The clean domain abstraction expected by all core application services.
Client	`CheckoutService`	Calls operations on the target interface without knowing low-level implementations.
Adaptee	`LegacyBankSystem`	Third-party or legacy class with an incompatible protocol, data types, or signatures.
Adapter	`LegacyBankAdapter`	Implements `IPaymentProcessor`, holds `LegacyBankSystem`, translates calls.

4. Full Production C# Implementation

Real-world payment provider conversion handling data transformation, error wrapping, and DI registration:

✅ **Complete Code: Clean Domain vs. Legacy Adaptee**

```
namespace PaymentAdapterPattern;

// 1. Domain Target Interface (What our app expects)
public interface IPaymentProcessor {
    void ProcessPayment(string invoiceId, decimal amount);
}

// 2. Third-Party Adaptee (Incompatible signatures, cents conversion, tokens)
public class LegacyBankSystem {
    public void SendWire(long centsAmount, string destinationAccount, string authToken) {
        Console.WriteLine($"[LegacyBank API] Wired {centsAmount} cents to {destinationAccount} (Auth:
{authToken})");
    }
}

// 3. The Adapter (Translates domain call into Adaptee semantics)
public class LegacyBankAdapter : IPaymentProcessor {
    private readonly LegacyBankSystem _legacyBank;
    private readonly string _accountNumber;
    private readonly string _apiKey;

    public LegacyBankAdapter(LegacyBankSystem legacyBank, string accountNumber, string apiKey) {
        _legacyBank = legacyBank;
        _accountNumber = accountNumber;
        _apiKey = apiKey;
    }

    public void ProcessPayment(string invoiceId, decimal amount) {
        // Translation 1: Format & validate decimal currency to integer cents
        long amountInCents = (long)(amount * 100m);

        // Translation 2: Append metadata to invoice reference
        Console.WriteLine($"[Adapter] Mapping Invoice {invoiceId} to Wire payload...");

        // Translation 3: Invoke the incompatible third-party method
        _legacyBank.SendWire(amountInCents, _accountNumber, _apiKey);
    }
}

// 4. Client Domain Service (Completely decoupled from LegacyBank)
public class OrderCheckoutService {
    private readonly IPaymentProcessor _paymentProcessor;

    public OrderCheckoutService(IPaymentProcessor paymentProcessor) {
        _paymentProcessor = paymentProcessor;
    }

    public void Checkout(string orderId, decimal total) {
        Console.WriteLine($"Starting checkout for Order #{orderId}");
        _paymentProcessor.ProcessPayment(orderId, total);
        Console.WriteLine($"Order #{orderId} settled successfully.\n");
    }
}

// 5. Execution Demo (Program.cs / IoC Setup)
public static class Program {
    public static void Main() {
        var legacyBank = new LegacyBankSystem();
        IPaymentProcessor adapter = new LegacyBankAdapter(legacyBank, "IBAN-EG-9988", "SECRET_AUTH_99");

        var checkout = new OrderCheckoutService(adapter);
        checkout.Checkout("INV-2026-4401", 250.75m);
    }
}
```

5. Real-World Use Cases in ASP.NET Core & .NET

🛠 Framework Implementation 1: Data Protection & Cache Adapters

ASP.NET Core provides `IDistributedCache` with methods like `GetAsync(string key)` returning `byte[]`. When developers work with typed objects or Redis clients (like `StackExchange.Redis`), they write an adapter layer that converts JSON strings or protocol buffers into the required raw byte streams.

🛠 Framework Implementation 2: Logging Infrastructure

Third-party logging providers (Serilog, NLog) do not directly implement Microsoft's `Microsoft.Extensions.Logging.ILogger`. Instead, packages like `Serilog.Extensions.Logging` provide an adapter class that translates `ILogger.Log()` into Serilog's proprietary message templates and sinks.

6. Trade-offs & Architecture Decision Matrix

✓ Pros (Benefits)

- **Adheres to Single Responsibility (SRP):** Keeps business rules separate from data conversion logic.
- **Adheres to Open/Closed (OCP):** Integrates brand-new legacy or external vendors without changing existing client classes.
- **Reusability:** Enables reuse of legacy code that otherwise would require a rewrite.

✗ Cons (Drawbacks)

- **Increased Indirection:** Introduces extra interfaces and wrapper classes into call stacks.
- **Maintenance Overhead:** If the adaptee introduces major signature changes, the adapter must be updated accordingly.

💡 Comparison: Adapter vs. Facade vs. Decorator vs. Proxy

• **Adapter:** Converts an existing, incompatible interface into an expected interface without altering functionality.

• **Facade:** Defines a new, simplified interface over a complex multi-class subsystem.

• **Decorator:** Keeps the same interface and enhances behavior dynamically.

• **Proxy:** Keeps the same interface to manage access, caching, or lazy loading.